ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

NAME: AKHILA POCHIMIREDDY

REG NO: 192425225

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: 11-11-2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:

https://github.com/120608akhila/DSA0117-object-oriented-programming-with-C-/tree/main/ASSIGNMENT_CO1

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom's Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

• The objects in the problem (card, passenger, journey, zone, fare table) and the data and

behaviour of each

• How the same problem would be written in a procedural style, and what changes in an object-oriented style

• The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system

• The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

1. Aim

To analyse the City Metro Smart-Card Fare System using object-oriented concepts and identify its objects, attributes, behaviours and correctness rules.

2. Problem analysis

The metro network has three zones: Inner, Middle and Outer. The basic fares are Rs 10, Rs 20 and Rs 30 respectively, according to the farthest zone travelled. A 20% surcharge applies during peak hours (8–10 AM and 5–8 PM). Students and senior citizens receive concessions. A journey is permitted only when at least Rs 50 remains on the smart card after deducting the fare. The design must avoid rounding errors and negative balances.

3. Objects, data and behaviour

Object	**Attributes (data)**	**Behaviour (operations)**
SmartCard	card ID, balance in paise	check balance, validate travel, deduct fare, recharge
Passenger	passenger ID, name, category	identify passenger type, provide applicable concession category

Journey	source, destination, zone, hour, calculated fare	validate journey, calculate journey fare
Zone	zone name, zone identifier	represent Inner, Middle or Outer zone
FareTable	base fares, surcharge percentage, concession rates	return base fare and calculate adjusted fare

Each object represents a real entity or responsibility. SmartCard owns the balance so that it can protect the minimum-balance rule. Passenger stores the category needed for concession decisions. Journey groups the details of one trip. Zone represents the fare region. FareTable centralises fare policy so that a fare revision is made in one place.

4. Procedural style and object-oriented style

In a procedural design, the program would use separate variables (such as balance, zone and hour) and free functions (such as calculateFare() and deductFare()). In an object-oriented design, related data and operations are grouped into classes such as SmartCard, Passenger, Journey and FareTable. The object-oriented design improves data protection, modularity and maintainability.

Procedural programming	**Object-oriented programming**
Uses free functions and separate data	Uses classes and objects
Data may be shared globally	Data can be private inside classes
Changes can affect multiple functions	Changes can be localised to a class or policy component
Reuse and extension need manual organisation	Supports inheritance, composition and polymorphism

5. OOP principles in the metro system

- Encapsulation: SmartCard keeps balance private and exposes controlled methods such as canTravel() and deduct(). This prevents unrelated code from directly making the balance negative.

- Abstraction: A gate asks whether a journey is allowed and requests fare deduction without needing to know every internal calculation step.
- Inheritance: Student and SeniorCitizen can be specialised passenger types derived from a Passenger base class if their common and distinct data/behaviour justify inheritance.
- Polymorphism: A virtual concession() or discountRate() operation can return different concession rules for NormalPassenger, Student and SeniorCitizen.

6. Rules every correct journey must satisfy

1. Select the fare according to the farthest zone travelled.
2. Apply the peak surcharge only from 08:00 inclusive to 10:00 exclusive, or from 17:00 inclusive to 20:00 exclusive.
3. Apply a concession only to eligible passenger categories and according to the officially defined rate.
4. Permit travel only when balance − fare is greater than or equal to Rs 50.
5. Never allow a negative balance or a deduction that violates the minimum balance.
6. Use exact integer paise for monetary arithmetic and validate all input values.

7. Alternative concept and complexity analysis

A purely procedural solution can store the card balance in a variable and use free functions to validate and deduct a fare. Both procedural and object-oriented implementations perform a fixed number of arithmetic and comparison operations for one journey.

```
bool canTravel(long long balance, long long fare) {
   return balance >= fare && balance - fare >= 5000;
}
void deductFare(long long &balance, long long fare) {
   if (canTravel(balance, fare)) balance -= fare;
}
```

Measure	Procedural	Object-oriented
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)

Change to minimum balance	May require updating multiple functions/constants if duplicated	Can be centralised in SmartCard/FarePolicy

Evaluation: The running time and per-card storage do not change asymptotically. The object-oriented approach is preferred because it groups the balance rule with the card and reduces the risk of inconsistent updates.

8. Conclusion

The metro fare problem naturally contains objects with distinct responsibilities. An object-oriented design is suitable because it protects the card balance, separates fare policy from journey data and supports future changes.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one

place only. Discuss:

• The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples

• #define, const and constexpr constants, and why constexpr is preferred for the fare table

• enum and enum class for zones, and why enum class is safer

• Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

1. Aim

To define fares, surcharge and limits in one place and use C++ tokens, constants and enumerations safely.

2. Five kinds of tokens

Token kind	Description	Examples from this system
Keywords	Reserved words with predefined meaning	int, class, if, return, constexpr
Identifiers	Programmer-defined names	balance, fare, SmartCard, calculateFare
Constants	Fixed literal or named values	10, 20, 30, MIN_BALANCE_PAISE
Operators	Symbols that perform operations	+ , -, *, /, ==, &&
Punctuators	Symbols that structure statements	; , { }, ( ), [ ]

3. Constant choices

#define is a preprocessor macro and performs text substitution. const declares a typed object that cannot be modified after initialisation. constexpr declares a value or function that can be evaluated at compile time when its expression permits it. constexpr is preferred for fixed fare policy values because it is typed, scoped and suitable for compile-time use.

4. enum and enum class

An unscoped enum can implicitly convert to an integer and its enumerators can enter the surrounding scope. enum class is strongly typed and its values are qualified, for example Zone::Inner. enum class therefore prevents accidental mixing of unrelated zone values and integers.

5. Naming conventions

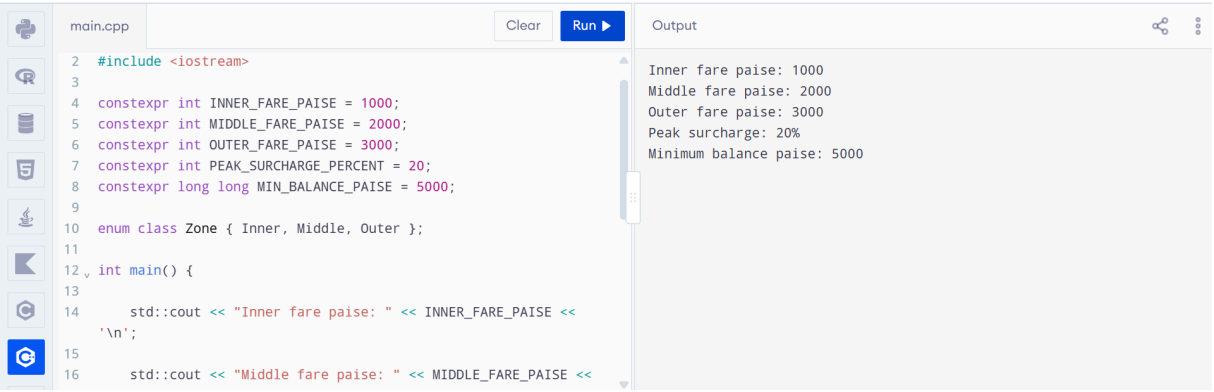
Identifiers must not start with a digit, must not contain spaces, cannot be C++ keywords and are case-sensitive. Use meaningful names such as totalFarePaise and minimumBalancePaise. A consistent convention such as PascalCase for class names and snake_case or camelCase for variables improves readability.

6. C++ program – constants and enum class

```
#include <iostream>
constexpr int INNER_FARE_PAISE = 1000;
constexpr int MIDDLE_FARE_PAISE = 2000;
constexpr int OUTER_FARE_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
enum class Zone { Inner, Middle, Outer };
int main() {
    std::cout << "Inner fare paise: " << INNER_FARE_PAISE << '\n';
    std::cout << "Middle fare paise: " << MIDDLE_FARE_PAISE << '\n';
    std::cout << "Outer fare paise: " << OUTER_FARE_PAISE << '\n';
    std::cout << "Peak surcharge: "
              << PEAK_SURCHARGE_PERCENT << "%\n";
    std::cout << "Minimum balance paise: "
              << MIN_BALANCE_PAISE << '\n';
    Zone selected = Zone::Outer;
    (void)selected;
    return 0;
}
```

7. Reference output

```
Inner fare paise: 1000
Middle fare paise: 2000
Outer fare paise: 3000
Peak surcharge: 20%
Minimum balance paise: 5000
```



8. Alternative concept and complexity analysis

If #define macros and an unscoped enum were used, the preprocessor would replace macro names before compilation. A macro has no C++ type or ordinary scope, so type errors and unexpected textual substitutions are easier to introduce. constexpr values have types and obey scope rules. An enum class requires qualified values and stronger type checking.

Property	**#define + enum**	**constexpr + enum class**
Type checking	Macro itself is not type checked	Typed constants and strongly typed zones
Scope	Macro has preprocessor scope	Normal C++ scope
Storage	Macro itself occupies no storage	Often no storage when used as compile-time constant
Runtime	No inherent runtime calculation	No inherent runtime calculation
Complexity	O(1) access	O(1) access

Conclusion: constexpr and enum class provide safer, clearer and more maintainable fare definitions.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

• The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
• Why money should be stored in integer paise and not as double rupees

• Type compatibility rules, and what the compiler does with int, double and bool values in the same expression

• The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

1. Aim

To select suitable C++ data types for fares, balances, time, zones and journey counts, and to avoid precision and overflow errors.

2. Data type selection

Value	Suggested type	Reason
Fare and balance	long long (integer paise)	Exact whole-paise arithmetic and large totals
Peak-hour flag	bool	Only true or false
Passenger category initial	char or enum class	Small category selection; enum class preferred
Hour	int	Whole-hour value 0–23
Zone	enum class Zone	Named, type-safe categories
Daily journey count	long long	Can represent millions and larger counts
Display-only rupees	double	Suitable for formatting/display, not authoritative money storage

3. Why money should be stored in integer paise

Floating-point values such as 0.1 cannot always be represented exactly in binary. Repeated addition or percentage calculations can produce tiny rounding errors. Integer paise represents every whole-paise amount exactly. For example, Rs 10.00 is stored as 1000 paise and Rs 36.00 as 3600 paise.

4. Type compatibility

In mixed expressions, C++ applies integral promotions and usual arithmetic conversions. A bool used in arithmetic is promoted to int (false becomes 0 and true becomes 1). An int combined with a double is converted to double. Integer division discards the fractional part before any later conversion to double.

5. sizeof and limits

```
#include <iostream>
```

```
#include <climits>

int main() {

   std::cout << "sizeof(bool) = " << sizeof(bool) << '\n';

   std::cout << "sizeof(char) = " << sizeof(char) << '\n';

   std::cout << "sizeof(int) = " << sizeof(int) << '\n';

   std::cout << "sizeof(long long) = " << sizeof(long long) << '\n';

   std::cout << "sizeof(float) = " << sizeof(float) << '\n';

   std::cout << "sizeof(double) = " << sizeof(double) << '\n';

   std::cout << "INT_MAX = " << INT_MAX << '\n';

   std::cout << "LLONG_MAX = " << LLONG_MAX << '\n';

   return 0;
}
```



The sizeof results and limits are implementation-dependent for some types. Compile and run this program on the actual system and enter the observed values here.

Type	Observed sizeof (bytes)	Range / note
bool	1	false or true
char	1	at least −127 to 127 or 0 to 255, depending on implementation
int	4	INT_MIN to INT_MAX from <climits>
long long	8	LLONG_MIN to LLONG_MAX from <climits>

float	4	approximately 6–7 decimal digits precision
double	8	approximately 15–16 decimal digits precision

6. Network collection and int capacity

At the whole-network scale, the brief gives 3,000,000 journeys per day. If every journey were Rs 30 plus a 20% peak surcharge, the maximum illustrative fare is Rs 36, so the total would be Rs 108,000,000 (10,800,000,000 paise). This exceeds the usual 32-bit signed int maximum of 2,147,483,647 paise. A long long is therefore appropriate for accumulated paise.

7. Alternative concept: double rupees

With double rupees, a fare such as Rs 0.10 is not exactly representable in binary. Repeatedly adding it may produce a value slightly different from the mathematical total. Both integer and double addition take O(1) time and both commonly occupy 8 bytes on current platforms, but exact storage size must be confirmed using sizeof.

```
#include <iostream>
#include <iomanip>
int main() {
  double total = 0.0;
  for (int i = 0; i < 1000000; ++i) total += 0.10;
  std::cout << std::fixed << std::setprecision(10) << total << '\\n';
  long long exactPaise = 0;
  for (int i = 0; i < 1000000; ++i) exactPaise += 10;
  std::cout << "Exact rupees: " << exactPaise / 100.0 << '\\n';
}
```

Reference mathematical total: Rs 100,000.00. Run the code and record the actual floating-point total and difference from Rs 100,000.00. Do not assume the measured last digits without running it.

Representation	Time per addition	Typical space per value	Precision
long long paise	O(1)	Often 8 bytes	Exact for whole paise until overflow
double rupees	O(1)	Often 8 bytes	Approximate binary floating point

Conclusion: Use long long paise for fare and balance calculations, and use double only for display or non-authoritative approximate calculations.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

• Integer division and integer-to-double conversion

• Comparison of signed and unsigned values, and the compiler warning it

produces • Narrowing conversions, and how brace initialisation (int x{3.7};)

prevents them • static_cast and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{
 int a = 7, b = 2;
 double c = a / b;
 double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';

 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';

 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';

 unsigned int u = 1;
 int s = -1;
```

```
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';

}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

1. Aim

To identify implicit conversions in C++ and use explicit casts and brace initialisation to prevent unintended behaviour.

2. Important conversions

- Integer division: 7 / 2 produces 3 because both operands are integers. Assigning the result to double does not restore the discarded fraction.
- Integer-to-double: static_cast<double>(7) / 2 produces 3.5 because the division is floating-point.
- char promotion: a char used in arithmetic is promoted to int. For example, 'C' + 1 produces the integer value 68 in the usual ASCII execution environment.
- int-to-bool: assigning 5 to bool produces true; assigning 0 produces false.
- Signed/unsigned comparison: when a negative int is compared with unsigned int, the signed operand may be converted to unsigned, producing a large positive value.
- Narrowing: int x{3.7}; is rejected because list initialisation prevents the implicit narrowing conversion.

3. Predict the output – write before execution

Predicted output (based on standard C++ conversion rules; record this before running the supplied program):

3 3.5

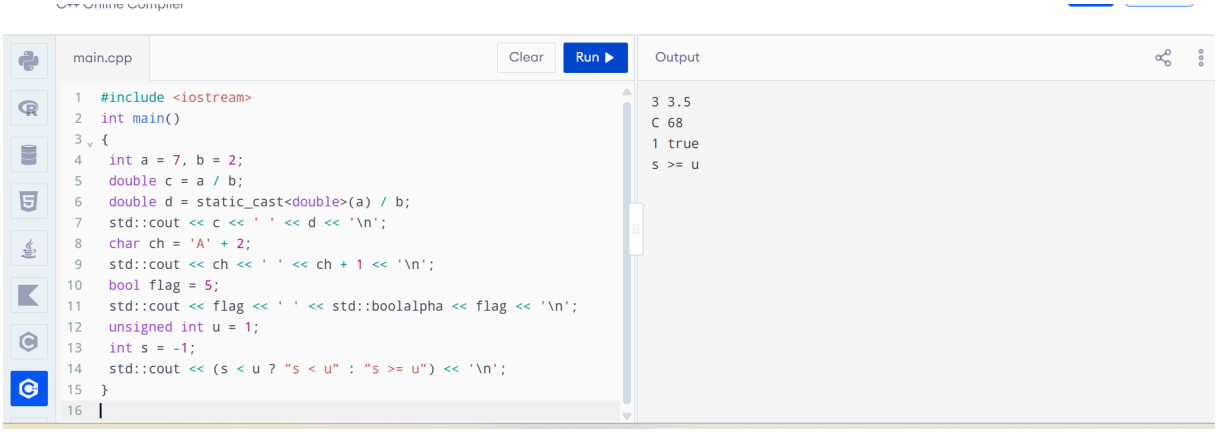
C 68

1 true

s >= u

4. Given program

```
#include <iostream>
int main()
{
 int a = 7, b = 2;
 double c = a / b;
 double d = static_cast<double>(a) / b;
 std::cout << c << ' ' << d << '\n';
 char ch = 'A' + 2;
 std::cout << ch << ' ' << ch + 1 << '\n';
 bool flag = 5;
 std::cout << flag << ' ' << std::boolalpha << flag << '\n';
 unsigned int u = 1;
 int s = -1;
 std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```



5. Explanation of each output line

- 3 3.5: a / b performs integer division and gives 3; assigning it to c converts 3 to 3.0. The explicit cast converts a to double before division, giving 3.5.
- C 68: 'A' + 2 is evaluated using integer promotion and produces the character code for C. Printing ch displays C. In ch + 1, ch is promoted to int and the result is 68 in ASCII.
- 1 true: bool flag = 5 converts non-zero int to true. The first output uses the default bool format (1); std::boolalpha prints true.
- s >= u: in the comparison s < u, s is converted to unsigned int. The converted value of −1 is a large unsigned value, so the comparison with 1 is false.

After execution, compare the actual output with the prediction and explain any compiler/platform-specific difference. The above is the expected result on a conventional system using ASCII and 32-bit unsigned int.

6. Corrected conversion examples

```
int whole{7};
double precise = static_cast<double>(whole) / 2;
char letter = static_cast<char>('A' + 2);
bool enabled = (5 != 0);
// int invalid{3.7}; // intentionally rejected: narrowing conversion
```

7. Alternative concept and complexity analysis

If all conversions were made explicit using static_cast and brace initialisation, the programmer would see where type changes occur. Narrowing list-initialisation would be diagnosed at compile time. Explicit casts do not normally change the runtime cost or memory use when they represent the same machine conversion.

Measure	**Implicit conversions**	**Explicit casts / brace initialisation**
Time	O(1) for each conversion	O(1) for each conversion
Memory	No additional storage required in ordinary expression conversions	No additional storage required in ordinary expression conversions
Readability	Shorter but less obvious	Makes intent visible
Error detection	Some surprises may remain unnoticed	Narrowing can be rejected at compile time

Conclusion: Use explicit casts where the intended type matters and brace initialisation for safer initialisation.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate
Write the expressions for the peak check, the surcharge and the concession.

Discuss:

• Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment

• The difference between x++ and ++x, and why i = i++ is a mistake

• The result of / and % with negative operands

• Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()

{

 int x = 5, y = 3;

 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';


 int z = x++ + ++y;

 std::cout << z << ' ' << x << ' ' << y << '\n';


 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';


 int k = 0;

 for (int i = 0; i < 10; i += 3) k += i;

 std::cout << k << '\n';

}
```

My predicted output (write before running):
For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

1. Aim

To construct correct fare expressions using C++ operators and understand precedence, associativity, increment, division and remainder.

2. Required fare expressions

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);

long long surcharge = peak
   ? baseFare * PEAK_SURCHARGE_PERCENT / 100
   : 0;

long long finalFare = baseFare + surcharge;
```

The peak expression uses relational operators before logical AND, and logical AND before logical OR. Parentheses make the two time windows explicit. The conditional operator selects the surcharge amount. Compound assignment (such as balance -= fare) subtracts and assigns.

3. Operator concepts

- x++ is post-increment: the expression yields the old value and then increments x. ++x is pre-increment: x is incremented before its value is used.
- i = i++ should not be used. It is an error-prone expression and, in modern C++, this particular expression has undefined behaviour because the modifications are unsequenced relative to each other.
- For integer division, C++ truncates toward zero. Thus −10 / 3 is −3 and −10 % 3 is −1.
- The expression (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is equivalent to hour >= 8 && hour < 10 || hour >= 17 && hour < 20 because && has higher precedence than ||. Parentheses improve readability.

4. Predict the output – write before execution

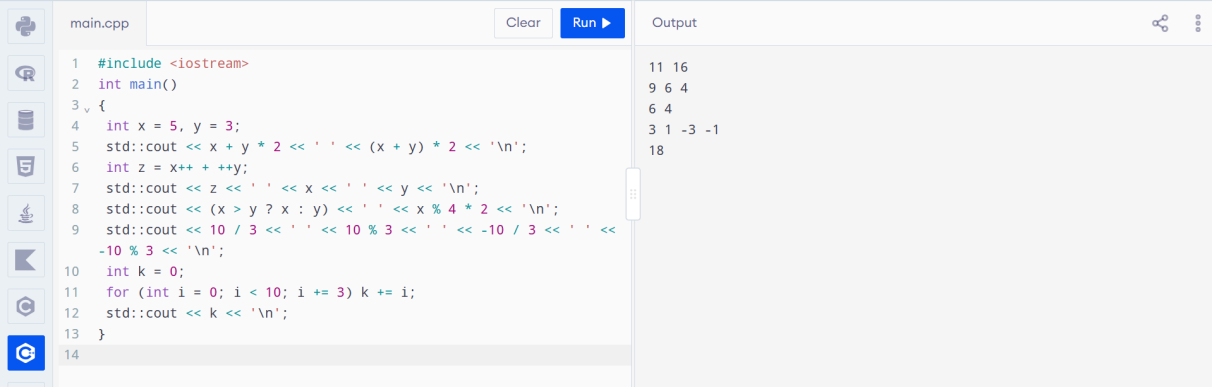
```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

5. Given program

```
#include <iostream>
int main()
{
 int x = 5, y = 3;
 std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';
 int z = x++ + ++y;
 std::cout << z << ' ' << x << ' ' << y << '\n';
 std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
 std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';
 int k = 0;
 for (int i = 0; i < 10; i += 3) k += i;
```

```
std::cout << k << '\n';
}
```

6. Explanation of output



- 11 16: multiplication has higher precedence than addition, so 5 + (3 × 2) = 11. Parentheses in (5 + 3) × 2 give 16.
- 9 6 4: x++ contributes old x = 5 and then x becomes 6. ++y first changes y from 3 to 4 and contributes 4. Therefore z = 5 + 4 = 9.
- 6 4: the conditional operator selects the larger of x=6 and y=4. Modulo and multiplication are evaluated left-to-right at the same precedence: (6 % 4) × 2 = 4.
- 3 1 -3 -1: integer division truncates toward zero; the remainder has the sign of the dividend.
- 18: the loop adds i values 0, 3, 6 and 9, giving 18.

7. Alternative concept: lookup table for peak hours

Instead of checking two ranges with && and ||, create a 24-element Boolean lookup table where valid peak hours are true. The lookup can be performed using peak[hour].

```
bool peak[24] = {};
for (int h = 8; h < 10; ++h) peak[h] = true;
for (int h = 17; h < 20; ++h) peak[h] = true;
// Validate hour is in [0, 23] before indexing.
bool isPeak = (hour >= 0 && hour < 24) ? peak[hour] : false;
```

Approach	Time per check	Extra space	Comment
Combined comparisons	O(1)	O(1)	Few comparisons; no table required
24-hour lookup table	O(1)	24 bool entries (typically 24 bytes, but confirm sizeof(bool))	Simple lookup; table must be initialised correctly

Conclusion: The range expression is compact and uses no lookup table. The table is useful when peak rules become more complex or configurable.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys. Discuss:

• When to use if–else, a switch on the zone, and the conditional operator

• How a for loop, a range-based for loop and a while loop could process the journeys, and which is best

• Where break and continue are needed (for example, skipping an invalid journey) • What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>
#include <iostream>


enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30 constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card


long long fare(Zone z, bool peak)
{
 // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add  // PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only
}
```

```
bool canTravel(long long balance, long long f)
{
 // TODO 2: Return true only if the balance left after paying the fare is at
least  // MIN_BALANCE_PAISE


}

int main()
{
 long long balance = 10000; // Rs 100.00 in paise
 struct Trip { Zone zone; bool peak; };
 Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};  for (const Trip& t : trips) {
 long long f = fare(t.zone, t.peak);
 if (canTravel(balance, f)) {
 balance -= f;
 std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
 << std::setfill('0') << f % 100 << std::setfill(' ');
 } else {
 std::cout << "Denied (low balance)";
 }
 std::cout << " balance Rs " << balance / 100 << '\n';
 }
}
```

Expected output:

Fare Rs 10.00 balance Rs 90

Fare Rs 36.00 balance Rs 54

Denied (low balance) balance Rs 54

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] = {1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

1. Aim

To design fare computation and daily journey processing using if–else, switch, loops, break and continue.

2. Choosing control structures

- if–else is suitable for validating time ranges, applying peak surcharge and checking whether a card has sufficient balance.
- switch is suitable for selecting the base fare from a finite set of zone values.
- The conditional operator is suitable for a short two-way value selection, but should not replace complex decision logic.
- A for loop is suitable when the number of journeys is known. A range-based for loop is convenient for iterating over a collection. A while loop is suitable when processing continues until a sentinel or input condition.
- continue skips the current invalid journey and proceeds to the next one. break exits a loop or switch when required.
- If no switch case matches, the default branch handles an invalid or unsupported zone. Without default, no matching branch executes.

3. SIMATS-style algorithmic pseudocode

```
ALGORITHM: COMPUTE_METRO_FARE
INPUT: zone, hour, cardBalance
CONSTANTS: INNER=1000, MIDDLE=2000, OUTER=3000
        SURCHARGE_PERCENT=20, MIN_BALANCE=5000
BEGIN
 IF hour < 0 OR hour > 23 THEN
```

```
 OUTPUT "Invalid hour"
      STOP
  ENDIF
  SWITCH zone
      CASE Inner: baseFare <- INNER
      CASE Middle: baseFare <- MIDDLE
      CASE Outer: baseFare <- OUTER
      DEFAULT:
          OUTPUT "Invalid zone"
          STOP
  ENDSWITCH
   peak <- ((hour >= 8 AND hour < 10) OR
 (hour >= 17 AND hour < 20))
  IF peak = TRUE THEN
      fare <- baseFare + (baseFare * SURCHARGE_PERCENT / 100)
  ELSE
      fare <- baseFare
  ENDIF


  IF cardBalance - fare >= MIN_BALANCE THEN
      cardBalance <- cardBalance - fare
      OUTPUT fare, cardBalance
  ELSE
      OUTPUT "Denied (low balance)"
  ENDIF
END
```

4. Complete the code – Task 6 supplied program

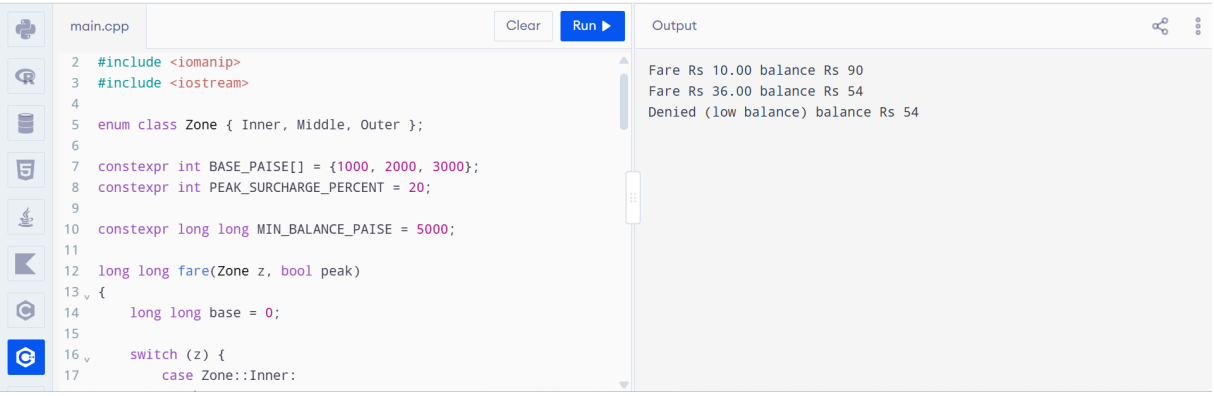
The following is the supplied program with TODO 1 and TODO 2 completed. The rest of the program structure is retained.

```
#include <iomanip>
#include <iostream>
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak)
{
   long long base = 0;
   switch (z) {
      case Zone::Inner:
         base = BASE_PAISE[0];
         Break;
      case Zone::Middle:
         base = BASE_PAISE[1];
         Break;
      case Zone::Outer:
         base = BASE_PAISE[2];
         Break;
      Default:
         return -1;
   }
   if (peak) {
      base += base * PEAK_SURCHARGE_PERCENT / 100;
   }
   return base;
}
bool canTravel(long long balance, long long f)
{
   return f >= 0 &&
         balance >= f &&          balance - f >= MIN_BALANCE_PAISE;
```

```
}
int main()
{
    long long balance = 10000;
    struct Trip {
        Zone zone;
        bool peak;
    };
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs "
                      << f / 100 << '.'
                      << std::setw(2)
                      << std::setfill('0')
                      << f % 100
                      << std::setfill(' ');
        }
        else {
            std::cout << "Denied (low balance)";
```

```
    }
    std::cout << " balance Rs "
              << balance / 100 << '\n';
  }
  return 0;
}
```

5. Expected output from the assignment



Calculation: Start with 10000 paise (Rs 100). Inner off-peak fare is 1000 paise, leaving 9000. Outer peak fare is 3000 + 20% = 3600 paise, leaving 5400. Middle fare is 2000 paise, but paying it would leave 3400 paise, below the required 5000, so the journey is denied.

6. Boundary check for peak hours

Time	Expected peak status	Reason
08:00	Peak	Start of morning range is inclusive
10:00	Off-peak	End of morning range is exclusive
17:00	Peak	Start of evening range is inclusive
20:00	Off-peak	End of evening range is exclusive

The supplied Trip structure does not contain an hour field. To test these boundaries, add a separate test program or extend a copy of the program with an hour member and the exact peak expression. Keep the original completed-code submission intact.

7. Alternative concept: fare array instead of switch

A constexpr fare array can map a zone index to a base fare. Both array indexing and a switch are O(1) for a fixed number of zones. The array occupies storage for its entries (the compiler

may place constexpr data in read-only storage). A fourth zone requires adding an enum value and a corresponding fare entry; a switch requires adding another case.

Approach	Time	Space	Fourth-zone change
switch	O(1)	O(1) additional	Add new enum value and case
fare array	O(1)	O(z) for z zones	Extend enum and array consistently

Conclusion: switch is clear for the three supplied zones. An indexed table may be easier to extend when zones and fare data grow.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two styles.

Discuss:

• A procedural version with separate variables and free functions

• An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()

• How the class prevents a negative balance

• Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

1. Aim

To implement the fare logic using procedural functions and a SmartCard class, and compare their safety and maintainability.

2. Procedural version

```
#include <iostream>
```

```
constexpr long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare) {
   return fare >= 0 && balance >= fare &&
          balance - fare >= MIN_BALANCE;
}
bool deductFare(long long &balance, long long fare) {
   if (!canTravel(balance, fare)) return false;
   balance -= fare;
   return true;
}
int main() {
   long long balance = 10000;
   long long fare = 3000;
   if (deductFare(balance, fare))
      std::cout << "Approved. Balance paise: " << balance << '\\n';
   else
      std::cout << "Denied. Balance paise: " << balance << '\\n';
}
```

3. Object-oriented version

```
#include <iostream>
class SmartCard {
private:
   long long balancePaise;
   static constexpr long long MIN_BALANCE_PAISE = 5000;
public:
   explicit SmartCard(long long initialBalance)
      : balancePaise(initialBalance >= 0 ? initialBalance : 0) {}
   long long getBalance() const {
      return balancePaise;
   }
   bool canTravel(long long farePaise) const {
      return farePaise >= 0 &&
             balancePaise >= farePaise &&
             balancePaise - farePaise >= MIN_BALANCE_PAISE;
   }
   bool deduct(long long farePaise) {
      if (!canTravel(farePaise))
         return false;
      balancePaise -= farePaise;
      return true;
```

```
    }
};
int main() {
    SmartCard card(10000);
    if (card.deduct(3000)) {
        std::cout << "Approved. Balance paise: "
                  << card.getBalance() << '\n';
    }
    else {

        std::cout << "Denied. Balance paise: "
                  << card.getBalance() << '\n';
    }
    return 0;
}
```



4. Comparison and balance protection

In the procedural version, the balance is passed by reference to a function. Other code may still modify the variable directly. In the SmartCard version, balancePaise is private. External code cannot change it directly and must use the class methods. deduct() checks the fare and minimum remaining balance before changing the stored value.

Measure	Procedural	SmartCard class
Time per validation/deduction	O(1)	O(1)
Space per card	O(1)	O(1)
Balance visibility	May be directly accessible	Private
Minimum balance change	Update shared rule/function	Update class policy constant or policy object

5. Alternative concept: public balance

If balancePaise were public, any part of the program could execute card.balancePaise = -500 or subtract a fare without checking the Rs 50 minimum. The time and space complexity

would remain O(1), but the invariant could be broken. Keeping the member private protects the rule.

6. Conclusion
Both versions have the same asymptotic performance. The SmartCard class is recommended because it encapsulates balance data and prevents uncontrolled updates.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

• Reading values with cin and checking cin.fail() when the operator types letters instead of numbers

• Clearing the error state with cin.clear() and discarding the bad input

• Formatting money with <iomanip> (setw, setfill, fixed, setprecision)

• Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

1. Aim
To validate console input, recover from invalid data and print fare reports using formatted output.

2. Safe input handling
When cin >> value receives letters where a number is expected, the stream enters a failure state. Check cin.fail(). Use cin.clear() to clear the error flags and cin.ignore() to discard the invalid characters up to the newline. Validate ranges after successful extraction.

3. C++ program – safe input and formatted report

```
#include <iostream>
#include <iomanip>
```

```
#include <limits>

int main() {

    int hour;

    std::cout << "Enter journey hour (0-23): ";

    while (!(std::cin >> hour) || hour < 0 || hour > 23) {

        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(
                std::numeric_limits<std::streamsize>::max(), '\n'
            );
        }

        std::cout << "Invalid hour. Enter 0-23: ";
    }

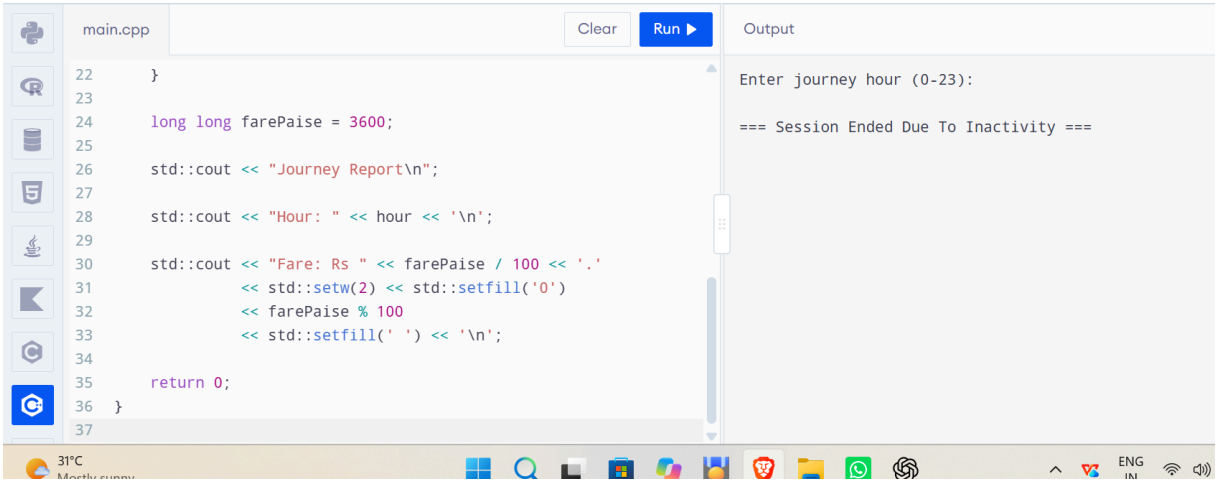
    long long farePaise = 3600;

    std::cout << "Journey Report\n";

    std::cout << "Hour: " << hour << '\n';

    std::cout << "Fare: Rs " << farePaise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << farePaise % 100
              << std::setfill(' ') << '\n';

    return 0;
}
```



4. Formatting tools

- setw(n) sets the minimum field width for the next formatted value.
- setfill('0') fills unused field positions with zero.
- fixed and setprecision(2) display a floating-point value with two digits after the decimal point. Integer paise formatting avoids floating-point rounding in the fare itself.

5. Alternative concept: getline and stoi

An alternative is to read each line using std::getline() and convert it using std::stoi() inside try/catch. Both approaches must inspect input length and validity. Processing a line of length L takes O(L) time, and the line buffer uses O(L) space.

```
#include <iostream>
#include <string>
int main() {
   std::string line;
   std::cout << "Enter hour: ";
   std::getline(std::cin, line);
   try {
      std::size_t used = 0;
      int hour = std::stoi(line, &used);
      if (used != line.size() || hour < 0 || hour > 23)
         throw std::invalid_argument("invalid hour");
      std::cout << "Valid hour: " << hour << '\\n';
   } catch (const std::exception&) {
      std::cout << "Invalid input\\n";
   }
}
```

Method	Invalid-input recovery	Time / space
cin >> with fail check	clear stream and discard remaining bad input	O(L) to discard L bad characters; stream buffer managed by library
getline + stoi	catch conversion exception; validate unconsumed characters	O(L) time and O(L) line buffer

Conclusion: Both methods are valid. cin extraction is convenient for simple numeric input; getline with stoi gives explicit control over complete input lines.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

• Normal journeys in each zone, peak and off-peak

• Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs

50 • Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A

small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

1. Aim

To implement and test fare calculation for normal, peak, boundary and invalid journeys.

2. Complete fare logic

```
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;

bool isPeak(int hour) {
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}

long long baseFare(Zone zone) {
   switch (zone) {
      case Zone::Inner:
```

```
            return 1000;

        case Zone::Middle:
            return 2000;

        case Zone::Outer:
            return 3000;

        default:
            return -1;
    }
}

long long calculateFare(Zone zone, int hour) {
    if (hour < 0 || hour > 23)
        return -1;

    long long base = baseFare(zone);

    if (base < 0)
        return -1;

    if (isPeak(hour)) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }

    return base;
}

bool canTravel(long long balance, long long fare) {
    return fare >= 0 &&
           balance >= fare &&
           balance - fare >= MIN_BALANCE_PAISE;
}

int main() {

    struct Test {
        Zone zone;
        int hour;
        long long balance;
```

```
    };

    Test tests[] = {
        {Zone::Inner, 11, 10000},
        {Zone::Middle, 8, 10000},
        {Zone::Outer, 10, 10000},
        {Zone::Outer, 17, 10000},
        {Zone::Inner, 20, 6000}
    };

    for (const auto& t : tests) {

        long long f = calculateFare(t.zone, t.hour);

        if (f < 0) {
            std::cout << "Invalid journey\n";
        }
        else if (canTravel(t.balance, f)) {
            std::cout << "Approved; fare paise=" << f
                      << "; balance after=" << t.balance - f << '\n';
        }
        else {
            std::cout << "Denied (low balance)\n";
        }
    }

    return 0;
}
```



3. Required test cases

Test	Input / condition	Expected result

1	Inner, off-peak 11:00	1000 paise (Rs 10.00)
2	Middle, peak 08:00	2400 paise (Rs 24.00)
3	Outer, 10:00	3000 paise (Rs 30.00), off-peak
4	Outer, 17:00	3600 paise (Rs 36.00), peak
5	Inner, 20:00	1000 paise (Rs 10.00), off-peak
6	Balance 6000 paise, fare 1000 paise	Allowed; exactly 5000 paise remains
7	Balance 5999 paise, fare 1000 paise	Denied; remaining balance would be below 5000
8	Hour −1 or 24	Invalid hour
9	Unknown zone value	Invalid zone / rejected
10	Negative recharge	Reject recharge; balance unchanged
11	Letters entered for hour	Input error; clear and recover

4. Five-journey hand calculation

Illustrative sequence with starting balance Rs 200.00: Inner off-peak Rs 10.00; Middle peak Rs 24.00; Outer off-peak Rs 30.00; Inner peak Rs 12.00; Middle off-peak Rs 20.00. Total fare = Rs 96.00. Ending balance = Rs 104.00, which is above the Rs 50 minimum.

Journey	Fare	Balance after journey
Start	—	Rs 200.00
1 Inner off-peak	Rs 10.00	Rs 190.00
2 Middle peak	Rs 24.00	Rs 166.00
3 Outer off-peak	Rs 30.00	Rs 136.00
4 Inner peak	Rs 12.00	Rs 124.00
5 Middle off-peak	Rs 20.00	Rs 104.00

5. Alternative concept: table-driven tests

Instead of writing separate checks manually, store each test input and expected fare in a test-case table and loop through it. For t test cases, total test execution is O(t), with O(t) storage for the table. A new boundary case can be added as another row without changing the test loop.

Conclusion: Table-driven tests improve repeatability and make boundary and invalid cases easier to add.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom’s Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

• Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys

agree, and by how much they differ

• Whether int overflows for the network total

• The compiler warnings you received with -Wall -Wextra and what each

meant • Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

1. Aim
To compare integer-paise and double totals, evaluate overflow risk and examine compiler warnings.

2. Journey-volume comparison
The brief gives 20,000 journeys per day at one station, 500,000 at one line and 3,000,000 across the whole network. The exact total depends on the journey mix and fare distribution. For a transparent upper-bound illustration, assume every journey costs the maximum original fare including peak surcharge: Rs 30 × 1.20 = Rs 36 = 3600 paise.

Scale	Journeys/day	Illustrative total rupees	Illustrative total paise
One station	20,000	Rs 720,000	72,000,000
One line	500,000	Rs 18,000,000	1,800,000,000
Whole network	3,000,000	Rs 108,000,000	10,800,000,000

These are upper-bound illustrative totals under the stated assumption, not measured system totals. Replace them with totals calculated from your actual journey dataset or test mix.

3. Integer overflow and precision
A typical 32-bit signed int has maximum value 2,147,483,647. The illustrative whole-network total of 10,800,000,000 paise exceeds that limit, so int is not sufficient. A signed long long is suitable for this scale. Double has a large range but can represent decimal amounts approximately rather than exactly.

4. Suggested comparison program
#include <iostream>

```cpp
#include <iomanip>

int main() {

  const long long journeys[] = {20000, 500000, 3000000};

  const long long farePaise = 3600;

  for (long long n : journeys) {

      long long totalPaise = n * farePaise;

      double totalRupees = static_cast<double>(n) * 36.0;

      std::cout << "Journeys: " << n
                << " integer paise: " << totalPaise
                << " double rupees: " << std::fixed
                << std::setprecision(2) << totalRupees << '\n';
  }

  return 0;
}
```



5. Compiler warnings

Compile with -Wall -Wextra. Record each warning actually shown by your compiler. Common examples include signed/unsigned comparison warnings, unused variables and narrowing or conversion warnings. The exact warning text depends on compiler and version.

6. Alternative concept and complexity analysis

Accumulator	At 3,000,000 × 3600 paise	Time/addition	Space
int	32-bit signed overflow	O(1)	Typically 4 bytes

long long	10,800,000,000 paise fits	O(1)	Typically 8 bytes
double rupees	Large enough range; approximate decimal representation	O(1)	Typically 8 bytes

Conclusion: Use long long paise for authoritative money totals. Record the actual compiler warnings and compare the totals produced by your local run.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

• The types, constants and classes you recommend and why

• How a fare revision will be carried out

• The limitations of your design

• Whether the design remains correct for the whole network
Your recommendation must be based on evidence from your own

experiments. Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

1. Aim
To recommend a final metro fare design based on correctness, maintainability and the required journey scale.

2. Recommended types, constants and classes
- Use long long for balances, fares and accumulated paise.
- Use int for hour values and long long for journey counts.
- Use bool for peak-hour status and enum class for zones and passenger categories.
- Use constexpr constants for fixed base fares, surcharge percentage and minimum balance.
- Use SmartCard to encapsulate balance and enforce the minimum-balance condition.

- Use Passenger or passenger-category policy to apply concessions, and Journey to hold trip details.
- Keep fare calculation in a FarePolicy/FareTable component rather than duplicating fare values across gate logic.

3. How a fare revision will be carried out

For a fixed fare revision, update the central fare constants or fare table, rebuild, run regression tests and deploy the tested version. Do not search and replace scattered literal values throughout the program. If fare changes become frequent, consider loading a validated configuration file at application startup.

4. Limitations

- The initial design supports only three zones and fixed fares.
- The assignment does not specify the exact concession percentage or precedence; the policy must be confirmed by the metro authority.
- A fixed minimum balance may not be appropriate for all card types or passes.
- A local in-memory card balance is not enough for a distributed production system; transaction persistence, concurrency, audit logs and secure communication would be required.

5. Whole-network suitability

For 3,000,000 journeys per day, per-journey fare computation remains O(1). A long long accumulator avoids the 32-bit int overflow shown in Task 10. Overall daily processing is O(n) for n journeys, which is appropriate because each journey must be processed once. Production deployment would additionally require scalable storage and transaction handling.

6. Alternative concept: read fare table from configuration file

A configuration file can store z zone fares. Loading it takes O(z) time and O(z) memory. The benefit is that fare changes can be made without recompiling the program, provided the configuration is validated and safely deployed. The compiled constexpr approach has no file-loading step and catches some errors at compile time, but every policy revision requires recompilation and redeployment.

Design	Start-up / update cost	Advantages	Risks
Compiled constexpr fare table	No runtime file load; rebuild for changes	Fast, typed, compile-time constants	Requires rebuild/deploy for policy change
Configuration file	O(z) read at start-up; O(z) storage	Policy can change without recompiling	Must validate missing, malformed or unsafe values

Conclusion: Retain the object-oriented design with integer paise and centralised fare policy for the current requirements. Extend the fare policy and classes when the metro introduces new rules.

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your

original solution. A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are

affected. Evidence Expected

A clear analysis of how the changed requirements affect the original

design. B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

Evidence Expected

An appropriate design response with working code.

D. Evidence-Based Defence – 2 Marks

Defend your decision using test results and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating tests and the changed requirements.

Changed requirement

The metro adds a fourth zone, Airport, with a fare of Rs 60; a 10% weekend discount that does not combine with the peak surcharge; and a monthly pass allowing unlimited journeys in two chosen zones.

A. Impact analysis (2 Marks)

Original design element	Required change
Zone enum	Add Zone::Airport
Base fare constants/table	Add Airport fare = 6000 paise
Peak expression	Retain weekday/time peak rule; ensure weekend discount replaces rather than combines with peak surcharge
Fare calculation	Add weekend input and a mutually exclusive discount/surcharge decision
SmartCard	Retain balance protection for pay-as-you-go journeys
Monthly pass	Add a Pass class containing validity and two selected zones
Journey validation	Check pass validity and zone eligibility before charging stored value

The most affected components are the zone definition, fare table, fare calculation policy and journey authorisation. The SmartCard minimum-balance invariant can remain for pay-as-you-go travel, but a valid monthly-pass journey need not deduct a per-journey fare if the metro's policy defines the pass as unlimited travel.

B. Evaluation of original design (2 Marks)

The original switch can absorb Airport by adding a new enum value and case, but every switch-based zone mapping must be updated consistently. A fare table indexed by the zone is easier to extend when the number of zones grows. The original SmartCard class can continue protecting the balance, but it does not represent pass validity or zone eligibility; a separate Pass class is a cleaner extension.

Worked examples, using the new requirements:

Example	Calculation	Final fare
Airport, weekday off-peak	Base fare	Rs 60.00
Airport, weekday peak	Rs 60 + 20%	Rs 72.00
Airport, weekend (peak time)	Weekend discount applies; no peak surcharge: Rs 60 − 10%	Rs 54.00
Inner, weekend	Rs 10 − 10%	Rs 9.00
Valid pass in selected Inner and Middle zones	Unlimited covered journey during pass validity	Rs 0.00 per covered journey

The brief does not state whether weekend discount applies to all passengers or how it interacts with passenger concessions. This design assumes the weekend discount replaces the peak

surcharge and is calculated from the base fare. Confirm any passenger-concession stacking policy with the faculty.

C. Adaptation / redesigned implementation (2 Marks)

The following example uses a fare table and separate Pass class. It assumes the weekend discount is 10% of the base fare and takes precedence over peak surcharge. The pass covers exactly two selected zones while valid.

```
#include <iostream>
#include <array>
#include <cstddef>

enum class Zone {
    Inner = 0,
    Middle = 1,
    Outer = 2,
    Airport = 3
};

constexpr std::array<long long, 4> FARE_PAISE = {
    1000, 2000, 3000, 6000
};
constexpr long long MIN_BALANCE_PAISE = 5000;
bool isPeak(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
long long calculateFare(Zone zone, int hour, bool weekend) {
    if (hour < 0 || hour > 23) {
        return -1;
    }
 std::size_t index = static_cast<std::size_t>(zone);
 if (index >= FARE_PAISE.size()) {
        return -1;
```

```
    }
 long long fare = FARE_PAISE[index];
if (weekend) {
        fare -= fare * 10 / 100;
    }
    else if (isPeak(hour)) {
        fare += fare * 20 / 100;
    }
    return fare;
}
class SmartCard {
private:
    long long balance;
public:
    explicit SmartCard(long long initial) {
        balance = (initial >= 0) ? initial : 0;
    }
    long long getBalance() const {
        return balance;
    }
    bool deduct(long long fare) {
        if (fare < 0 || balance < fare ||
            balance - fare < MIN_BALANCE_PAISE) {
            return false;
        }
        balance -= fare;
        return true;
    }
};
class Pass {
private:
```

```
    Zone firstZone;
    Zone secondZone;
    bool valid;
public:
    Pass(Zone a, Zone b, bool isValid) {
        firstZone = a;
        secondZone = b;
        valid = isValid;
    }
    bool covers(Zone z) const {
        return valid &&
               (z == firstZone || z == secondZone);
    }
};
int main() {
    SmartCard card(20000);
    Pass monthlyPass(Zone::Inner, Zone::Middle, true);
    Zone trip1 = Zone::Inner;
    if (monthlyPass.covers(trip1)) {
        std::cout << "Pass accepted; fare Rs 0.00" << std::endl;
    }
    else {
        long long f = calculateFare(trip1, 9, false);
        if (card.deduct(f)) {
            std::cout << "Card charged paise: " << f << std::endl;
        }
        else {
            std::cout << "Travel denied" << std::endl;
        }
    }
    long long airportWeekend =
```

```
    calculateFare(Zone::Airport, 18, true);
  std::cout << "Airport weekend fare paise: "
            << airportWeekend << std::endl;
  return 0;
}
```

Implementation note: This is a simplified teaching model. A production monthly pass also needs start/end dates, renewal, expiry validation, identity/card binding and secure storage. The example's bool valid stands in for a validity check.



D. Evidence-based defence (2 Marks)

Decision: MODIFY the original design rather than discard it. Retain SmartCard encapsulation and integer-paise arithmetic; replace the small switch mapping with an indexed fare table; introduce a separate Pass class; and make weekend discount and weekday peak surcharge mutually exclusive.

Test	Input	Expected result
1	Airport weekday off-peak	6000 paise
2	Airport weekday peak	7200 paise
3	Airport weekend at 18:00	5400 paise; no peak surcharge
4	Inner weekend	900 paise
5	Valid pass, Inner zone	Pass accepted; no fare deduction
6	Valid pass, Airport zone	Pass does not cover zone; calculate normal fare
7	Invalid hour 24	Rejected
8	Card deduction leaves Rs 49.99	Rejected; minimum balance rule preserved

Trade-off: The revised design adds a fare table and Pass class, which slightly increases program structure and storage but improves extensibility. Fare lookup and pass-zone checks

remain O(1) because the number of zones and pass zones is fixed. The design is easier to extend than adding more scattered switch statements and special-case conditions.

Conclusion: The modified design supports Airport, weekend discount and two-zone monthly passes while preserving exact paise arithmetic and SmartCard balance protection.

MARK DISTRIBUTION

Task Bloom's Level Marks

Task 1 – Object-Oriented Analysis K5 – Evaluate 4

Task 2 – Tokens and Constants K5 – Evaluate 3

Task 3 – Data Types and Compatibility K5 – Evaluate 4

Task 4 – Implicit Conversions K5 – Evaluate 4

Task 5 – Operators and Precedence K5 – Evaluate 4

Task 6 – Control Structures K5 – Evaluate 4

Task 7 – Procedural vs Object-Oriented K5 – Evaluate 4

Task 8 – Input and Output K5 – Evaluate 4

Task 9 – Implementation & Testing K5 – Evaluate 4

Task 10 – Expectation vs Execution K5 – Evaluate 4

Task 11 – Final Recommendation K5 – Evaluate 4

Subtotal 42

Task 12 – Day-4 Design Challenge K5 – Evaluate 8

TOTAL 50

Submission Requirements

• GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.

• Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.